

**Examen
Semestre 6
Deuxième session
2020/2021**

Licence 3

EC : INFO0601

Intitulé : Systèmes d'exploitation : concepts avancés

Responsable : Cyril RABAT

Durée de l'épreuve : 2h00

Nombre de page(s) : 5

RAPPEL

Aucun document, outil de calcul ou de communication autorisé. Sauf indications contraires, la gestion d'erreur ne doit pas être réalisée et les inclusions ne doivent pas être spécifiées. Dans les portions de code demandées, déclarez les variables utilisées. Toute réponse doit être justifiée.

Sujet : M. Cyril Rabat (20 points)

Nous souhaitons développer un jeu de bataille navale en temps réel, multi-joueurs et en réseau. L'application répartie est constituée de trois applications distinctes : un *coordinateur*, un *client* et un *serveur*.

Le but du jeu. Chaque joueur possède un nombre de bateaux placés sur un plateau de jeu qui est une grille avec une largeur et une hauteur données : chaque case peut contenir plusieurs bateaux. Le *client* est l'application exécutée par chaque joueur. Il affiche les cases du plateau de jeu, les bateaux du joueur et il permet au joueur de réaliser des actions. Au début du jeu, le joueur place ses bateaux sur le plateau de jeu et attend le début de la partie. Celle-ci démarre une fois que le nombre de joueurs est atteint et qu'ils sont tous prêts. Le jeu se déroule en tour par tour : à chaque tour, tous les joueurs peuvent réaliser simultanément une action qui peut être soit un tir, soit un déplacement de bateau. Pour tirer, le joueur choisit une case : si un ou plusieurs bateaux sont présents, il sont détruits. Pour déplacer un bateau, le joueur en sélectionne un puis clique sur une case adjacente (haut, bas, droite ou gauche). Un joueur a perdu lorsqu'il ne possède plus de bateau. Le jeu s'arrête lorsqu'il ne reste plus qu'un seul joueur.

Les applications. Le *serveur* permet aux *clients* de se connecter au jeu depuis des machines distantes. Nous utiliserons les communications en mode connecté. À chaque connexion d'un *client*, le *serveur* crée un processus fils. Ce dernier récupère les informations sur le jeu depuis un segment de mémoire partagée (SMP). Un tableau de sémaphores (TdS) est utilisé pour la synchronisation. Le but du *coordinateur*, exécuté sur la même machine, est d'initialiser le jeu. En paramètre, il prend la largeur et la hauteur du plateau de jeu, ainsi que le nombre de joueurs et le nombre de bateaux pour chaque joueur. Il crée le SMP et le TdS.

1°) **Généralités (1 point).** Représentez l'architecture de l'application en précisant les interactions entre les différentes applications, les outils IPC et en représentant les communications réseau.

2°) **Le segment de mémoire partagée (8 points).** Le SMP contient toutes les informations partagées entre les joueurs et est accessible par les fils du serveur. Les données sont la configuration du jeu (dimensions, nombre de joueurs et de bateaux) et les informations sur chaque joueur (le nombre de bateaux restant et les positions des bateaux).

a) Donnez le contenu du SMP sous la forme d'un schéma ; expliquez l'intérêt de chaque donnée, le type utilisé et la taille.

b) Pour pouvoir consulter ou modifier les données du SMP, nous souhaitons utiliser une structure en C nommée `segment_t`. Donnez cette structure et les autres structures dont vous auriez éventuellement besoin.

La fonction `creer_segment` crée le SMP et retourne une structure `segment_t` permettant de le mapper. Ses paramètres sont : `cle` est la clé IPC du segment, `l` et `h` sont respectivement la largeur et la hauteur de la grille, `nbJoueurs` est le nombre de joueurs et `nbBateaux` est le nombre de bateaux de chaque joueur. La signature est la suivante :

```
segment_t creer_segment(key_t cle, int l, int h, int nbJoueurs, int nbBateaux)
```

c) Expliquez les actions réalisées par cette fonction.

d) Donnez le code de la fonction `creer_segment`.

3°) **Les sockets (5 points).** Les communications entre les *clients* et les fils du *serveur* sont en mode connecté. Le *serveur* écoute sur un numéro de port spécifié dans l'application par la constante `PORT_ECOUTE`.

a) Donnez le code principal du serveur permettant de créer la socket et de créer un fils pour chaque connexion. Nous ne nous intéresserons pas à l'arrêt du serveur qui tournera en boucle. Le code du fils est situé dans une fonction `void fils(...)` dont vous donnerez uniquement la signature.

b) En supposant que l'adresse IP de la machine sur laquelle s'exécute le *serveur* est spécifiée en argument du programme *client*, donnez la portion de code du *client* permettant d'établir une connexion avec le *serveur*.

c) Combien de numéro de port sont utilisés sur le serveur ? Combien de sockets ? À quels endroits doit-on faire des appels à `close` ?

d) Nous souhaitons que le serveur s'arrête dès qu'il reçoit `CRTL + C` et qu'il se mette en attente de la fin de ses fils. Expliquez quels mécanismes doivent être mis en place et comment modifier votre code précédent.

4°) **Le tableau de sémaphores (6 points)**. Le *coordinateur* crée le TdS qui sera utilisé pour les différentes synchronisations entre les applications.

a) Nous souhaitons utiliser un seul sémaphore pour vérifier que tous les joueurs attendus sont bien connectés et s'assurer qu'il n'y ait pas plus du nombre de joueurs autorisés. Expliquez les différentes actions à réaliser (initialisation, opérations généralisées, *etc.*) par le *coordinateur* et les fils du serveur.

b) Avec votre solution, que se passe-t-il si un joueur tente de se connecter alors que le nombre maximum de joueurs est déjà atteint ? Si ce n'est pas déjà fait, proposez une solution pour que le fils du *serveur* s'arrête correctement.

c) Pour éviter de créer un fils inutilement si le nombre de joueurs est atteint, proposez une solution au niveau du *serveur*.

d) Un tour s'arrête dès que tous les joueurs ont réalisé leur action (soit un tir, soit un déplacement). En utilisant un ou plusieurs sémaphores, expliquez comment gérer les tours : un seul coup par joueur, passer au tour suivant, *etc.* Vous préciserez les sémaphore(s) utilisé(s), leur initialisation, les opérations généralisées, *etc.*

e) Les actions sont traitées immédiatement, pas uniquement à la fin du tour. Expliquez les problèmes de synchronisation que cela pose. Proposez une solution en utilisant un nombre minimal de sémaphores pour résoudre ces problèmes : les joueurs ne doivent pas être bloqués inutilement pendant leur tour.

f) Donnez la portion de code permettant au *coordinateur* de créer le TdS (clé IPC définie par la constante `CLE_SEM`) et de l'initialiser. Vous pourrez utiliser des variables ou des constantes pour faciliter l'écriture du code : vous donnerez alors leur valeur et leur signification.

g) Lorsque le *serveur* démarre, il vérifie que le TdS existe. Si ce n'est pas le cas, il se met en attente et vérifie à nouveau, jusqu'à ce que le tableau soit effectivement créé. Donnez la portion de code permettant de réaliser ce test.

Annexes : quelques en-têtes de fonctions C

```
int      accept(int sockfd, struct sockaddr *adresseClient, socklen_t *taille);
unsigned int alarm(unsigned int nb_secondes);
int      atexit(void (*procedure)(void));
int      bind(int fd, struct sockaddr *adresse, socklen_t longueur);
int      close(int fd);
int      connect(int sockfd, struct sockaddr *adresseServeur, socklen_t taille);
int      execve(const char *fichier, char *const argv[], char *const envp[]);
void     exit(int statut);
int      fcntl(int fd, int commande, ...);
pid_t    fork();
key_t    ftok(char *chemin, int proj_id);
pid_t    getpid();
pid_t    getppid();
int      getpeername(int sockfd, struct sockaddr *adresse, socklen_t *taille);
int      getsockname(int sockfd, struct sockaddr *adresse, socklen_t *taille);
uint32_t htonl(uint32_t hote_long);
uint16_t htons(uint16_t hote_court);
const char *inet_ntop(int famille, const void *source, char *destination, socklen_t taille);
int      inet_pton(int famille, const char *source, void *destination);
int      kill(pid_t pid, int numero_signal);
int      listen(int sockfd, int taille);
off_t    lseek(int fd, off_t offset, int point_depart);
int      lstat(const char *chemin, struct stat *tampon);
void     *memcpy(void *destination, const void *source, size_t taille);
void     *memset(void *tampon, int caractere, size_t nombre_elements);
int      mkfifo(const char *nomFichier, mode_t mode);
int      msgctl(key_t cle, int commande, struct msqid_ds *buf)
int      msgget(key_t cle, int options);
int      msgrcv(int msqid, struct msgbuf *msg, size_t taille, long type, int options);
int      msgsnd(int msqid, struct msgbuf *msg, size_t taille, int options);
uint32_t ntohl(uint32_t reseau_long)
uint16_t ntohs(uint16_t reseau_court);
int      open(const char *chemin, int options, mode_t mode);
int      pause(void);
int      pipe(int tube[2]);
ssize_t  recvfrom(int sockfd, void *message, size_t taille, int flags,
                  struct sockaddr *adresse_source, socklen_t *taille_adresse);
ssize_t  read(int fd, void *tampon, size_t taille);
int      semctl(int semid, int semnum, int commande, union senum args);
int      semget(key_t cle, int nbSems, int options);
int      semop(int semid, struct sembuf *sops, unsigned nsops);
ssize_t  sendto(int sockfd, const void *message, size_t taille, int flags,
                  const struct sockaddr *adresse_dest, socklen_t taille_adresse)
void     *shmat(int shmid, const void *shmaddr, int options);
int      shmctl(key_t cle, int commande, struct shmid_ds *buf);
int      shmdt(const void *shmaddr);
int      shmget(key_t cle, int taille, int options);
int      sigaction(int num, const struct sigaction *action, struct sigaction *old_action);
int      sigaddset(sigset_t *ensemble, int numero_signal);
int      sigdelset(sigset_t *ensemble, int numero_signal);
int      sigemptyset(sigset_t *ensemble);
int      sigfillset(sigset_t *ensemble);
int      sigismember(sigset_t *ensemble, int numero_signal);
int      sigpending(sigset_t *ensemble);
int      sigprocmask(int comment, const sigset_t *ensemble, sigset_t *ancien);
int      sigqueue(pid_t pid, int numero_signal, const union sigval valeur);
int      sigsuspend(const sigset_t *ensemble);
int      sigwaitinfo(const sigset_t *ensemble, siginfo_t *info);
int      sigtimedwait(const sigset_t *set, siginfo_t *info, const struct timespec *timeout);
unsigned int sleep(unsigned int nombre_secondes);
```

```

int      socket(int domaine, int type, int protocole);
int      socketpair(int domaine, int type, int protocol, int sv[2]);
int      unlink(const char *nomFichier);
pid_t    wait(int *statut);
pid_t    waitpid(pid_t pid, int *statut, int options);
ssize_t  write(int fd, const void *tampon, size_t taille);

```

Constantes/macros associées à des paramètres ou des champs de structures

```

cle (msgget, shmget, semget) : IPC_PRIVATE
commande (fcntl) : F_GETFL, F_SETFL
commande (msgctl, shmctl) : IPC_RMID, IPC_STAT, IPC_SET
commande (semctl) : IPC_RMID, IPC_STAT, IPC_SET, IPC_GETVAL, IPC_GETALL, IPC_SETVAL, IPC_SETALL
comment (sigprocmask) : SIG_BLOCK, SIG_UNBLOCK, SIG_SET
domaine (socket, socketpair) : AF_LOCAL, AF_UNIX, AF_INET, AF_INET6, AF_PACKET
famille (inet_pton, inet_ntop) : AF_INET, AF_INET6
mode (mkfifo) : O_RDONLY, O_WRONLY, O_CREAT, O_EXCL, S_IRWXU, S_IRUSR, S_IWUSR, S_IXUSR...
mode (open) : S_IRWXU, S_IRUSR, S_IWUSR, S_IXUSR...
options (fcntl, open) : O_RDONLY, O_WRONLY, O_RDWR, O_CREAT, O_EXCL, O_TRUNC, O_APPEND
options (msgget, semget, shmget) : IPC_CREAT, IPC_EXCL, S_IRWXU, S_IRUSR, S_IWUSR, S_IXUSR...
options (msgrcv) : IPC_NOWAIT, MSG_EXCEPT
options (msgsnd) : IPC_NOWAIT, MSG_NOERROR
options (shmat) : SHM_RDONLY, SHM_RND
options (waitpid) : WNOHANG, WUNTRACED
point_depart (lseek) : SEEK_SET, SEEK_CUR, SEEK_END
protocole (socket, socketpair) : IPPROTO_TCP, IPPROTO_UDP
sa_flags (champ de struct sigaction) : SA_ONESHOT, SA_NOMASK, SA_SIGINFO
sa_handler (champ de struct sigaction) : SIG_IGN, SIG_DFL
sem_flag (champ de struct sembuf) : IPC_NOWAIT, SEM_UNDO
taille (inet_ntop) : INET_ADDRSTRLEN, INET6_ADDRSTRLEN
type (socket, socketpair) : SOCK_STREAM, SOCK_DGRAM

```

Macros sur **statut** de wait, waitpid : WIFEXITED, WEXITSTATUS, WIFSIGNALED, WTERMSIG, WIFSTOPPED, WSTOPSIG

Quelques structures C

<pre> struct sigaction { void (*sa_handler) (int); void (*sa_sigaction) (int, siginfo_t*, void*); sigset_t sa_mask; int sa_flags; }; union senum { int val; struct semid_ds *buf; unsigned short *array; }; struct siginfo_t { int si_signo; int si_code; sigval_t si_value; pid_t si_pid; ... }; struct msqid_ds { struct ipc_perm msg_perm; time_t msg_stime; time_t msg_rtime; time_t msg_ctime; msgnum_t msg_qnum; msglen_t msg_qbytes; pid_t msg_lspid; pid_t msg_lrpid; }; </pre>	<pre> struct in_addr { uint32 s_addr; }; struct timespec { long tv_sec; long tv_nsec; }; struct semid_ds { struct ipc_perm sem_perm; time_t sem_otime; time_t sem_ctime; unsigned short sem_nsems; }; struct shmid_ds { struct ipc_perm msg_perm; size_t shm_segsz; time_t shm_atime; time_t shm_dtime; time_t shm_ctime; pid_t shm_cpid; pid_t shm_lpid; shmatt_t shm_nattch; }; struct sockaddr_in { sa_family_t sin_family; in_port_t sin_port; uint32_t sin_addr; struct in_addr sin_addr; }; </pre>	<pre> struct sembuf { unsigned short sem_num; short sem_op; short sem_flg; }; struct sockaddr_in6 { sa_family_t sin6_family; in_port_t sin6_port; uint32_t sin6_flowinfo; struct in6_addr sin6_addr; uint32_t sin6_scope_id; }; struct sockaddr_un { sa_family_t sun_family; char sun_path[108]; }; struct in_addr6 { unsigned char s_addr6[16]; }; union sig_val_t { int sival_int; void *sival_ptr; }; struct timeval { long tv_sec; long tv_usec; }; </pre>
---	--	--